



Librerías de Python para Algoritmos de IA - Desarrollo Completo

INTRODUCCIÓN

En la actualidad, el desarrollo de la inteligencia artificial (IA) es uno de los campos más relevantes en el ámbito de la tecnología y la informática. Muchos de los algoritmos fundamentales de IA, como los de búsqueda, aprendizaje automático y optimización, se implementan y usan habitualmente en Python, debido a la versatilidad y riqueza de sus librerías. Este trabajo presenta una recopilación de las librerías de Python más importantes para implementar y experimentar con los algoritmos de IA que conforman la Unidad 1 del curso, acompañando cada uno con ejemplos de uso, documentación oficial y las funciones clave para una utilización eficiente en proyectos académicos y profesionales.

ALGORITMOS DE BÚSQUEDA (DFS, BFS, A*, etc.)

Pensamiento: Imagina que estás en un laberinto y quieres encontrar la salida.

- **DFS (Búsqueda en Profundidad):** Como si fueras tocando una pared y la sigues hasta el final, aunque te alejes.
 - *Ejemplo real:* Buscar las llaves revisando cada habitación a fondo antes de pasar a la siguiente.
- **BFS (Búsqueda en Anchura):** Revisas todas las habitaciones cercanas primero, luego las siguientes.
 - *Ejemplo real:* Buscar a alguien en un centro comercial: revisas toda la planta 1, luego la 2, etc.
- **A*: Usas un mapa y sabes más o menos dónde está lo que buscas.**
 - *Ejemplo real:* Usar Google Maps que sabe tu posición y tu destino.

ALGORITMOS DE OPTIMIZACIÓN E INTELIGENCIA COLECTIVA

Pensamiento: Cómo encontrar la mejor solución entre muchas posibilidades.

- **Algoritmos Genéticos:** Como criar perros: tomas los mejores padres (soluciones) y mezclas sus características para criar mejores hijos.

- *Ejemplo real:* Diseñar el coche más aerodinámico probando miles de formas.
- **Colonia de Hormigas:** Como las hormigas que dejan feromonas para marcar el mejor camino a la comida.
 - *Ejemplo real:* Optimizar rutas de reparto.

ALGORITMOS DE JUEGOS Y TOMA DE DECISIONES

- **Minimax:** Pensar "si yo hago esto, el otro hará aquello, entonces yo responderé con esto otro..."
 - *Ejemplo real:* Jugar al ajedrez pensando varios movimientos adelante.
- **Descenso del Gradiente:** Como bajar una montaña con los ojos cerrados: sientes con los pies hacia dónde va hacia abajo y vas en esa dirección.
 - *Ejemplo real:* Ajustar poco a poco los parámetros de un modelo hasta que funcione bien.

APRENDIZAJE AUTOMÁTICO (MACHINE LEARNING) - LOS MÁS IMPORTANTES

Pensamiento: Enseñar a la computadora a reconocer patrones.

A. SUPERVISADO (con ejemplos etiquetados)

- **Regresión Lineal:** Predecir números continuos.
 - *Ejemplo real:* Predecir el precio de una casa basándote en su tamaño y ubicación.
- **KNN (K Vecinos Más Cercanos):** "Dime con quién andas y te diré quién eres".
 - *Ejemplo real:* Si tus 3 amigos más cercanos les gusta el fútbol, probablemente a ti también.
- **SVM (Máquinas de Soporte Vectorial):** Dibujar la línea que mejor separa dos grupos.
 - *Ejemplo real:* Separar correos spam de los normales.

B. NO SUPERVISADO (sin etiquetas)

- **K-medias:** Agrupar cosas similares.
 - *Ejemplo real:* Agrupar clientes por comportamientos de compra similares.

C. REDES NEURONALES (como el cerebro)

- **Perceptrón Simple:** Una neurona artificial que decide SÍ/NO.
 - *Ejemplo real:* Decidir si hace suficiente sol para ir a la playa.
- **MLP (Perceptrón Multicapa):** Muchas neuronas trabajando juntas.
 - *Ejemplo real:* Reconocer si en una foto hay un gato o un perro.

MODELOS PROBABILÍSTICOS

- **Redes Bayesianas:** Relaciones de causa-efecto con probabilidades.
 - *Ejemplo real:* "Si llueve (causa), probablemente el suelo esté mojado (efecto)"
- **Modelos Ocultos de Markov:** Para secuencias donde no ves todo directamente.
 - *Ejemplo real:* Adivinar el tiempo que hará basándote solo en si la gente lleva paraguas o no.

Tipo de algoritmo	Representación visual más común	Librerías recomendadas
DFS, BFS, A*, Minimax	Árboles o grafos	networkx, graphviz
GA, ACO, Gradiente	Curvas de convergencia, trayectorias	matplotlib, plotly
Regresión, KNN, SVM, MLP	Fronteras de decisión, curvas de ajuste	seaborn, mlxtend, matplotlib
Bayesianas, HMM, Restricciones	Diagramas probabilísticos / redes de dependencias	pgmpy, graphviz

¿PARA QUÉ SIRVE TODO ESTO EN LA PRÁCTICA?

Aplicaciones reales que usas cada día:

1. Google Maps → A*, Búsqueda de caminos
2. Recomendaciones de Netflix/Amazon → KNN, Redes Bayesianas
3. Reconocimiento facial del móvil → Redes Neuronales (MLP)
4. Detección de spam → SVM
5. Asistentes de voz (Siri, Alexa) → Modelos Ocultos de Markov
6. Sugerencias de escritura → Algoritmos de lenguaje

¿CÓMO SE RELACIONA TODO ESTO CON PYTHON?

Python es como el lenguaje universal para estas técnicas. Las librerías son cajas de herramientas:

- **Scikit-learn** → La caja de herramientas principal (tiene de todo)
- **NetworkX** → Herramientas especiales para laberintos y conexiones
- **TensorFlow/PyTorch** → Herramientas profesionales para redes neuronales
- **DEAP** → Herramientas para algoritmos evolutivos

ALGORITMOS DE BÚSQUEDA

1. Búsqueda en profundidad (DFS)

DFS explora un camino lo más profundo posible antes de retroceder (backtracking) → antes de probar otro camino llega al final del que está.

Ventaja → usa poca memoria

Desventaja → puede quedarse atascado en caminos largos sin salida

Librería: NetworkX

Documentación: <https://networkx.org/documentation/stable/>

<https://networkx.org/documentation/stable/reference/algorithms/traversal.html>

Función específica: `dfs_edges`, `dfs_tree` (g, source, Depth_limit=None, ...), `dfs_preorder_nodes`

Documentación

función: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.traversal.depth_first_search.dfs_edges.html

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.traversal.depth_first_search.dfs_tree.html

Descripción: NetworkX proporciona implementaciones eficientes de DFS para grafos. La función `dfs_edges` retorna un generador de aristas en orden DFS, mientras que `dfs_tree` retorna un árbol orientado construido con DFS.

Ejemplo de uso:

```
import networkx as nx
```

```
G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])
edges = list(nx.dfs_edges(G, source=1))
# Resultado: [(1, 2), (2, 4), (2, 5), (1, 3), (3, 6)]
```

2. Búsqueda en anchura (BFS)

Librería: NetworkX

Documentación: <https://networkx.org/documentation/stable/>

<https://networkx.org/documentation/stable/reference/algorithms/traversal.html>

Función específica: `bfs_edges`, `bfs_tree`, `bfs_layers`

Documentación

función: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.traversal.breadth_first_search.bfs_edges.html

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.traversal.breadth_first_search.bfs_tree.html

Descripción: Similar a DFS pero explora todos los nodos por niveles, antes de ir al más profundo. Útil para encontrar caminos más cortos en grafos no ponderados.

Ventaja → encuentra la solución más corta (si el costo de cada paso es igual).

Desventaja → consume más memoria.

Ejemplo de uso:

```
python
import networkx as nx
```

```
G = nx.Graph()
G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)])
edges = list(nx.bfs_edges(G, source=1))
# Resultado: [(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)]
```

3. Búsqueda voraz (Greedy Search)

Elige en cada paso la opción que parece mejor en ese momento (según una heurística) → Elige el camino más corto visible sin mirar más allá.

Ventaja → Rápido

Desventaja → puede no encontrar la mejor solución global

Librería: No existe librería estándar específica

Razón: La búsqueda voraz es más un patrón de diseño que un algoritmo específico. Se implementa comúnmente como parte de otros algoritmos o de forma personalizada según el problema.

Alternativa: Se puede implementar usando PriorityQueue de heapq:

```
import heapq
```

```
def greedy_search(start, goal, heuristic):
    frontier = []
    heapq.heappush(frontier, (heuristic(start, goal), start))
    visited = set()
    while frontier:
        _, current = heapq.heappop(frontier)
        if current == goal:
            return True
        visited.add(current)
        # Expandir nodos vecinos...
```

4. Algoritmo A*

Librería: Python-astar ("astar" en PyPI)

Documentación: <https://pypi.org/project/astar/>

Función específica: AStar (clase que se hereda)

Descripción: Combina coste real con heurística para encontrar el camino óptimo. Requiere una función heurística → Combina la rapidez de la búsqueda voraz con la precisión de la búsqueda por costo (heurística + costo acumulado).

Ventaja → suele encontrar el camino óptimo rápidamente

Desventaja → depende de una buena heurística

Ejemplo de uso:

```
import networkx as nx
import math
```

```
def distancia_euclidiana(node1, node2):
    x1, y1 = node1
    x2, y2 = node2
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)
G = nx.grid_2d_graph(5, 5)
camino = nx.astar_path(G, (0, 0), (4, 4), distancia_euclidiana)
```

5. Algoritmo Minimax

Qué hace: busca la mejor jugada suponiendo que el oponente también juega perfectamente

Ejemplo: juegos como el ajedrez o el tres en raya

Ventaja: permite estrategias óptimas en juegos competitivos

Desventaja: muy costoso si el árbol de jugadas es grande

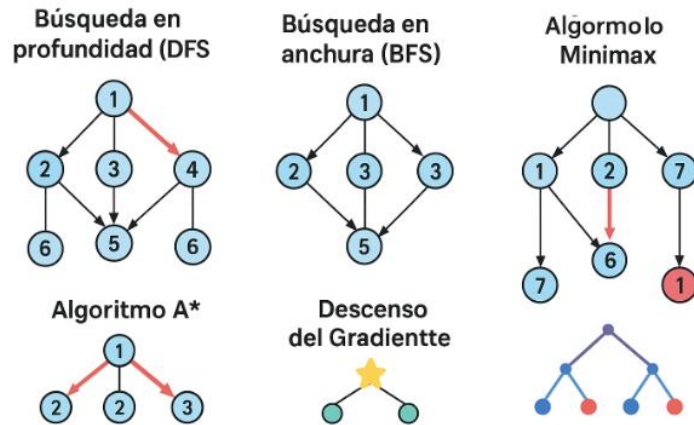
Librería: No existe librería estándar específica

Razón: Altamente dependiente del juego específico. Se implementa según las reglas del juego.

Implementación típica:

```
def minimax(estado, profundidad, es_maximizador):
    if profundidad == 0 or estado.es_terminal():
        return estado.evaluar()

    if es_maximizador:
        mejor_valor = -float('inf')
        for movimiento in estado.movimientos_posibles():
            valor = minimax(estado.aplicar_movimiento(movimiento),
                            profundidad-1, False)
            mejor_valor = max(mejor_valor, valor)
        return mejor_valor
    else:
        mejor_valor = float('inf')
        for movimiento in estado.movimientos_posibles():
            valor = minimax(estado.aplicar_movimiento(movimiento),
                            profundidad-1, True)
            mejor_valor = min(mejor_valor, valor)
        return mejor_valor
```

ALGORITMOS DE OPTIMIZACIÓN

6. Algoritmos Genéticos (GA)

Librería: PyGAD

Documentación: <https://pygad.readthedocs.io/>

Función específica: se utilizan clases como `pygad.GA` y se configura `Fitness_func`, `crossover_type`, etc

Documentación función: <https://deap.readthedocs.io/en/master/api/algo.html>

Librería: DEAP

Documentación: <https://github.com/DEAP/deap>

Función específica: usa `creator.create`, `toolbox.register`, `algorithms.varAnd` como parte de su interfaz.

Descripción: Framework completo para algoritmos evolutivos que permite personalizar operadores genéticos, funciones de fitness y estrategias de selección. Imitan la evolución biológica (selección, cruce, mutación) para mejorar soluciones. Ej: busca la mejor forma de organizar horarios, rutas o parámetros.

Ventaja → exploran muchas soluciones

Desventaja → lentos para problemas pequeños

Ejemplo de uso:

```
from deap import base, creator, tools, algorithms
import random
```

```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)
```

```
toolbox = base.Toolbox()
toolbox.register("attr_bool", random.randint, 0, 1)
toolbox.register("individual", tools.initRepeat, creator.Individual,
                 toolbox.attr_bool, 100)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

7. Optimización por Colonia de Hormigas (ACO)

Imita el comportamiento de hormigas que buscan comida y dejan feromonas.

Encuentra rutas óptimas en mapas o redes.

Ventaja → encuentra buenos caminos en problemas complejos

Desventaja → necesita ajustar muchos parámetros

Librería: ACOPY

Documentación: <https://acopy.readthedocs.io/en/latest/>

Función específica: la clase principal es ACOPY (y módulos internos)

Limitación: Documentación limitada, librería poco mantenida.

Otra librería: aco (PyPI)

Documentación: <https://pypi.org/project/aco>

Comentario: Disponible para ACO, aunque quizá menos extendida que las de ML tradicionales.

Ejemplo básico:

```
from aco import ACO, Graph

def distancia(c1, c2):
    # Calcular distancia entre ciudades
    pass

# Configurar el problema del viajante
ciudades = [...] # Lista de coordenadas
graph = Graph(ciudades, distancia)
aco = ACO(ant_count=10, generations=100)
camino, coste = aco.solve(graph)
```

8. Descenso del Gradiente

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: SGDClassifier, SGDRegressor

Documentación función: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

Descripción: Implementa descenso de gradiente estocástico para clasificación y regresión. Ajusta los parámetros de un modelo reduciendo el error paso a paso, bajando por la “pendiente” del error.

Ventaja → simple y efectivo

Desventaja → puede quedarse en mínimos locales

Ejemplo de uso:

```
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
```

```
X = [[0, 0], [1, 1]]
y = [0, 1]
clf = make_pipeline(StandardScaler(),
                    SGDClassifier(max_iter=1000, tol=1e-3))
clf.fit(X, y)
```

9. Propagación de Restricciones

Librería: python-constraint

Documentación: <https://pypi.org/project/python-constraint/>

Clase específica: Problem, BacktrackingSolver, RecursiveBacktrackingSolver

Documentación función: <https://pypi.org/project/python-constraint/>

Descripción: Resuelve problemas de satisfacción de restricciones (CSP). Busca valores que cumplan todas las condiciones (restricciones) de un problema.

Ejemplos: Resolver Sudokus o planificar tareas sin conflictos

Ventaja → garantiza soluciones válidas

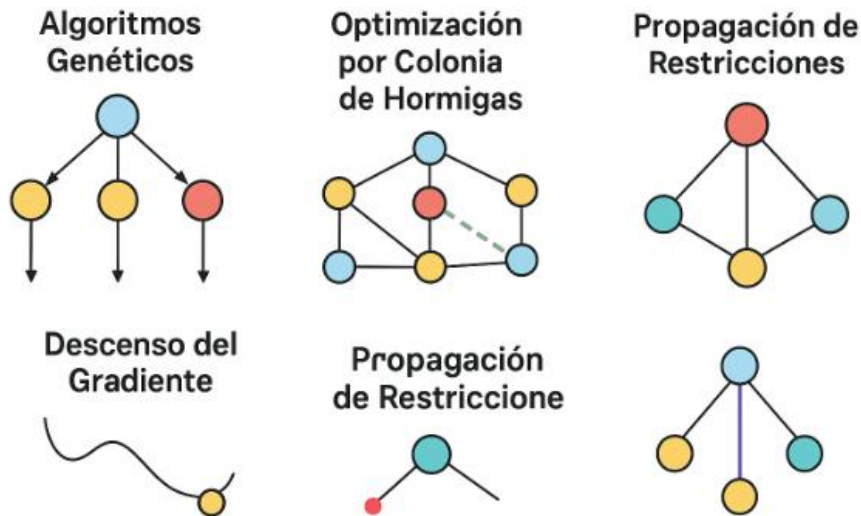
Desventaja → puede ser lento en problemas grandes

Ejemplo de uso:

```
from constraint import Problem, RecursiveBacktrackingSolver

problem = Problem(RecursiveBacktrackingSolver())
problem.addVariables(["a", "b"], [1, 2, 3])
problem.addConstraint(lambda a, b: a != b, ["a", "b"])
soluciones = problem.getSolutions()
```

Algoritmos de optimización



ALGORITMOS DE APRENDIZAJE AUTOMÁTICO (ML)

10. Regresión Lineal

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: LinearRegression

Documentación función: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

Descripción: Implementa regresión lineal ordinaria por mínimos cuadrados.

Busca una línea recta que se ajuste a los datos para predecir valores.

Ejemplo: predecir el precio de una casa según su tamaño

Ventaja → fácil de interpretar

Desventaja → sólo modela relaciones lineales

Ejemplo de uso:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
X = [[0, 1], [1, 2], [2, 3], [3, 4]]
```

```
y = [1, 2, 3, 4]
```

```
reg = LinearRegression().fit(X, y)
predicciones = reg.predict([[4, 5]])
```

11. K Vecinos Más Cercanos (KNN)

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: KNeighborsClassifier, KNeighborsRegressor

Documentación función: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

Descripción: Algoritmo basado en instancias para clasificación y regresión. Clasifica o predice según los vecinos más cercanos a un punto.

Ejemplo: recomendar películas similares a las que te gustan

Ventaja → simple, sin entrenamiento

Desventaja → lento con muchos datos

Ejemplo de uso:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target)
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
precision = knn.score(X_test, y_test)
```

12. Máquinas de Vectores de Soporte (SVM)

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: SVC, SVR, LinearSVC

Documentación función: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

Descripción: Algoritmo de aprendizaje supervisado para clasificación y regresión. Encuentra la línea o frontera que mejor separa las clases

Ejemplo: clasificar correos como “spam” o “no spam”

Ventaja → alta precisión

Desventaja → más difícil de ajustar

Ejemplo de uso:

```
from sklearn import svm
from sklearn.datasets import make_classification

X, y = make_classification(n_features=4, random_state=0)
clf = svm.SVC(kernel='linear')
clf.fit(X, y)
predicciones = clf.predict([[0, 0, 0, 0]])
```

13. K-medias

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: KMeans

Documentación función: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>

Descripción: Algoritmo de clustering no supervisado que particiona datos en k clusters. Agrupa datos en K grupos según similitud.

Ejemplo: segmentar clientes en grupos por comportamiento.

Ventaja → rápido y fácil

Desventaja → hay que elegir K manualmente.

Ejemplo de uso:

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=300, centers=4, random_state=0)
kmeans = KMeans(n_clusters=4, random_state=0).fit(X)
etiquetas = kmeans.labels_
centroides = kmeans.cluster_centers_
```

14. Redes Bayesianas

Librería: pgmpy

Documentación: <https://pgmpy.org/>

Clase específica: BayesianModel, VariableElimination

Documentación función: <https://pgmpy.org/models/bayesiannetwork.html>

Descripción: Librería para trabajar con modelos gráficos probabilísticos. Modelan relaciones probabilísticas entre variables.

Ejemplo: diagnóstico médico (si tienes fiebre y tos, probabilidad de gripe)

Ventaja → maneja incertidumbre

Desventaja → necesita muchos datos para entrenarse bien.

Ejemplo de uso:

```
from pgmpy.models import BayesianModel
from pgmpy.factors.discrete import TabularCPD
```

```
modelo = BayesianModel([('D', 'G'), ('I', 'G'), ('G', 'L'), ('I', 'S')])
# Definir tablas de probabilidad condicional...
```

15. Modelos Ocultos de Markov (HMM)

Librería: hmmlearn

Documentación: <https://hmmlearn.readthedocs.io/>

Clase específica: hmm.GaussianHMM, hmm.MultinomialHMM, hmm.GMMHMM

Documentación función: <https://hmmlearn.readthedocs.io/en/latest/api.html>

Librería: PyHHMM

Documentación: <https://phhmm.readthedocs.io/>

Clase específica: pyhhmm.HHMM (modelo de HMM heterogéneo)

Descripción: Implementa modelos ocultos de Markov para secuencias de datos. Modela procesos donde no observas los estados directamente, pero sí sus efectos.

Ejemplo: Reconocimiento de voz o texto

Ventaja → potente para secuencias temporales

Desventajas → necesita muchos datos para entrenarse bien

Ejemplo de uso:

```
from hmmlearn import hmm
import numpy as np
```

```
X = np.array([[1], [2], [3], [1], [2], [3]])
modelo = hmm.GaussianHMM(n_components=3, covariance_type="full")
modelo.fit(X)
estados_ocultos = modelo.predict(X)
```

16. Perceptrón Simple

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: Perceptron

Documentación función: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Perceptron.html

Descripción: Clasificador lineal simple que aprende pesos para características de entrada. Red neuronal con una sola capa que separa clases lineales

Ejemplo: distinguir entre dos tipos de objetos (si se pueden separar por una línea)

Ventaja → simple y rápido

Desventaja → no puede resolver problemas no lineales

Ejemplo de uso:

```
from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification
```

```
X, y = make_classification(n_features=2, n_redundant=0, random_state=0)
clf = Perceptron(tol=1e-3, random_state=0)
clf.fit(X, y)
predicciones = clf.predict([[0, 0]])
```

17. Perceptrón Multicapa (MLP)

Librería: Scikit-learn

Documentación: <https://scikit-learn.org/stable/>

Clase específica: MLPClassifier, MLPRegressor

Documentación función: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Descripción: Red neuronal feed-forward para clasificación y regresión. Red neuronal con varias capas ocultas, aprende relaciones no lineales complejas.

Ejemplo: reconocimiento de imágenes, voz o texto.

Ventaja → muy potente

Desventaja → requiere mucho entrenamiento y datos

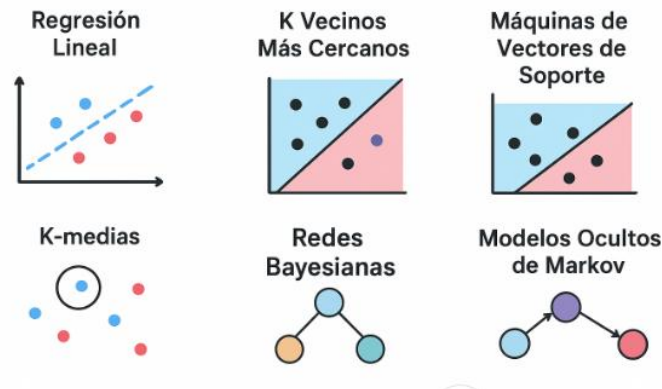
Ejemplo de uso:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
```

```
X, y = make_classification(n_features=4, random_state=0)
```

```
clf = MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=1000)
clf.fit(X, y)
predicciones = clf.predict([[0, 0, 0, 0]])
```

Algoritmos de aprendizaje automático



Resumen General

Librerías Principales Identificadas:

1. **NetworkX** - Algoritmos de grafos y búsqueda
2. **Scikit-learn** - Aprendizaje automático clásico
3. **DEAP** - Algoritmos evolutivos y genéticos
4. **pgmpy** - Redes Bayesianas y modelos probabilísticos
5. **hmmlearn** - Modelos Ocultos de Markov
6. **python-constraint** - Satisfacción de restricciones

Algoritmos sin librería específica

- Búsqueda voraz (se implementa según el problema)
- Minimax (dependiente del juego)
- ACO (librerías disponibles pero con documentación limitada)

Recomendaciones:

- Para algoritmos de ML clásico: **Scikit-learn**
- Para algoritmos de grafos: **NetworkX**
- Para algoritmos evolutivos: **DEAP**
- Para redes neuronales avanzadas: **PyTorch/TensorFlow** (no cubiertos aquí pero muy utilizados)